

PROJETO: INTEGRA+



Curso: Engenharia da Computação

Disciplina: PI: Projeto e Implementação de Aplicativos **Código:** 13132

Turma: 101 ou 102

Professor: Fernando Luiz de Almeida Silveira

Nome do Grupo: INTEGRA+

1. Felipe Natali Amaral – RA:24792566
2. José Ricardo Padovan Ferreira – RA: 24010692
3. Pedro Pimentel Rodrigues– RA: 24023362
4. Pedro Guimarães Peruchi – RA: 24019888
5. Tiago Aureliano Lança Rodriguez – RA: 25004196

Campinas, 1º semestre de 2026

1. Ideação do Projeto

1.1 Problema

Segundo o Instituto Brasileiro de Geografia e Estatística, por meio do Censo Demográfico 2022 (dados divulgados em 2025), cerca de 1 milhão de pessoas nascidas em outros países viviam no Brasil em 2022, o maior número registrado desde 1980, representando um aumento de aproximadamente 70% em relação ao Censo de 2010.

Grande parte desse fluxo migratório é composta por imigrantes sul-americanos, majoritariamente de países de língua hispânica. Com o crescimento desse movimento migratório, intensificam-se também os desafios relacionados à integração social e econômica dessa população.

Muitos imigrantes recém-chegados enfrentam dificuldades para reconstruir suas vidas em um novo contexto cultural, jurídico e linguístico. Entre as principais dificuldades identificadas estão a regularização documental, o acesso ao mercado de trabalho formal e o acesso a serviços essenciais, como saúde e assistência pública.

Além disso, informações fundamentais para a permanência regular no país encontram-se frequentemente dispersas em diferentes canais, o que dificulta o acesso organizado, claro e confiável. Como consequência, muitos imigrantes permanecem em situação de vulnerabilidade social, enfrentando obstáculos para acessar direitos básicos, regularizar sua situação migratória e ingressar no mercado de trabalho de forma digna.

1.2 Público-Alvo

Imigrantes sul-americanos que se encontram no território brasileiro em situação de vulnerabilidade social, especialmente aqueles que não possuem domínio da língua portuguesa, com foco principal em adultos em idade ativa para o trabalho, entre 18 e 55 anos, de qualquer nível de escolaridade, residentes em centros urbanos com acesso à internet.

1.3 SETOR DA SOCIEDADE

O projeto enquadra-se no Terceiro Setor por estar voltado ao apoio social e comunitário, sem fins lucrativos, atuando na área de assistência e inclusão social.

1.4 ODS Relacionadas

- ODS 3 – Saúde e Bem-estar
 - ODS 8 – Trabalho Decente e Crescimento Econômico
 - ODS 10 – Redução das Desigualdades
-

1.5 Tipo de Impacto Gerado

O tipo de impacto gerado pelo projeto é social, já que promove a inclusão de imigrantes na sociedade brasileira por meio da ampliação do acesso à informação clara, organizada e acessível sobre regularização documental, oportunidades de trabalho e serviços públicos de saúde.

1.6 Descrição Geral

O aplicativo tem como objetivo a integração do imigrante no Brasil, oferecendo suporte nas áreas de empregabilidade, regularização documental e acesso à saúde.

A proposta é centralizar informações essenciais em um único ambiente acessível, facilitando a adaptação do usuário ao novo contexto social e econômico.

1.7 Funcionalidades Principais

• Funcionalidade 1: Cadastro e autenticação de usuário

Permite que o imigrante crie uma conta, realize login seguro e acesse funcionalidades personalizadas da plataforma.

• Funcionalidade 2: Buscar vagas de trabalho

Possibilita pesquisar vagas por cidade ou região, além de visualizar detalhes das oportunidades disponíveis.

• Funcionalidade 3: Guia de regularização documental

Disponibiliza informações organizadas em formato de passo a passo e links oficiais para solicitação.

• Funcionalidade 4: Localização de serviços de saúde

Permite consultar pontos de atendimento próximos à localidade do usuário (UBS e

hospitais), além de apresentar orientações sobre a utilização do SUS e os direitos básicos de acesso à saúde.

- **Funcionalidade 5: Cadastro de vagas por empresas**

Permite que empresas criem contas e publiquem vagas no sistema.

1.8 Como a Tecnologia Resolve o Problema

A tecnologia proposta soluciona o problema ao centralizar, em uma única plataforma digital, as principais informações relacionadas às dificuldades enfrentadas por esse grupo social.

Por meio de uma aplicação web de fácil acesso e navegação intuitiva, o sistema permitirá que o usuário consulte conteúdos em sua língua nativa, reduzindo barreiras linguísticas.

A plataforma facilitará o acesso a vagas de trabalho cadastradas no site, orientações sobre regularização migratória e informações sobre o direito à saúde pública gratuita no Brasil.

Dessa forma, a solução tecnológica organiza e simplifica o acesso a informações essenciais, promovendo maior autonomia, inclusão social e integração do imigrante à sociedade brasileira.

1.9 VIABILIDADE TÉCNICA

A escolha da plataforma web permite maior abrangência e acessibilidade, alcançando um número maior de imigrantes, independentemente do tipo de dispositivo utilizado.

Por não exigir download ou instalação, a solução facilita o acesso, especialmente para usuários que possuem celulares com pouca memória ou recursos limitados.

Além disso, o aplicativo será desenvolvido com design responsivo, garantindo adequado funcionamento tanto em computadores quanto em dispositivos móveis.

1.10 Estrutura Técnica

O projeto utilizará a seguinte arquitetura para o desenvolvimento do MVP:

- **Front-end:** interface web responsiva, projetada para funcionar em múltiplos

- dispositivos, com navegação intuitiva e acessível ao público-alvo.
- **Back-end:** responsável pelo processamento de dados, gerenciamento das requisições e comunicação com o banco de dados.
 - **Banco de Dados:** armazenará e estruturará informações sobre usuários, empresas cadastradas, vagas de trabalho, documentação e serviços de saúde pública.
 - **API:** permitirá integração com serviços externos, principalmente para localização de unidades de saúde.

A estrutura garante que a aplicação seja funcional, escalável e de fácil manutenção, atendendo às necessidades do público-alvo de forma eficiente.

1.11 Viabilidade no Semestre

O projeto é viável dentro do período determinado, pois prioriza o desenvolvimento das funcionalidades essenciais do MVP, como:

- cadastro de usuários imigrantes;
- cadastro de empresas;
- publicação e busca de vagas por cidade e profissão;
- guia de documentação;
- consulta de unidades de saúde.

Inicialmente, o MVP não incluirá recursos mais complexos, como candidatura direta às vagas ou notificações automáticas, concentrando-se na criação e disponibilização das oportunidades de forma organizada.

Dessa forma, com a adequada divisão de tarefas entre os integrantes do grupo, é possível construir e concluir o MVP dentro do prazo estabelecido.

1.12 VALOR GERADO PELO PROJETO

O projeto gera valor social ao facilitar a inserção de imigrantes na sociedade brasileira, promovendo maior autonomia, inclusão social e melhor qualidade de vida aos usuários.

2. Protótipo da Solução

O protótipo do sistema foi desenvolvido com o objetivo de representar de forma visual a estrutura inicial da aplicação web INTEGRA+, priorizando a usabilidade e a acessibilidade para o público-alvo (imigrantes).

A interface foi projetada de maneira simples e intuitiva, permitindo a fácil utilização da plataforma por qualquer pessoa. O protótipo apresenta as principais telas do sistema, refletindo as funcionalidades definidas na proposta inicial do projeto, como cadastro de usuários, busca de vagas de trabalho, guia de documentação e acesso a serviços de saúde.

O protótipo não contempla animações, interações avançadas com botões ou transições de tela, tendo como foco a organização estrutural e a experiência básica de navegação. Da mesma forma, a funcionalidade de troca de idioma não foi prototipada visualmente, pois representa as mesmas telas, alterando apenas o idioma dos textos.

O protótipo foi desenvolvido na plataforma Figma, onde é possível visualizar todas as telas do sistema, bem como o fluxo de navegação representado por conexões entre elas.

Link de acesso ao protótipo:

<https://www.figma.com/integra+>

2.1 Telas iniciais

Esse conjunto de telas representa a primeira funcionalidade da aplicação, sendo responsável pelo processo de login e cadastro de usuários e a tela inicial do projeto.

No protótipo desenvolvido no Figma, também é possível visualizar telas complementares, como recuperação/alteração de senha.



Figura 1 tela inicial

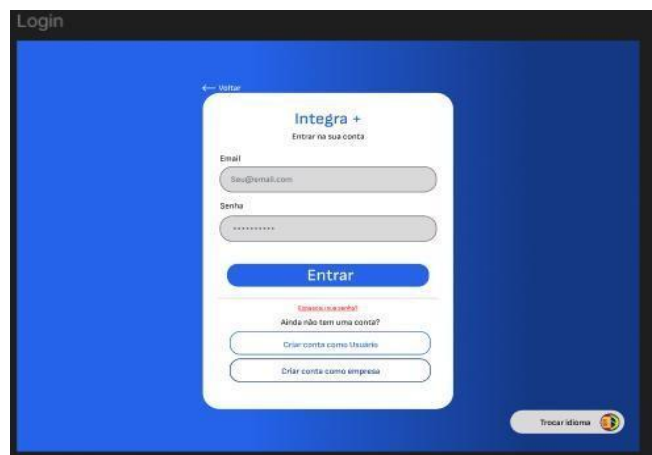


Figura 2 tela de login

Figura 3 cadastro usuário

Figura 4 cadastro empresa

2.2 Tela principal

A tela principal centraliza as funcionalidades do sistema, permitindo acesso rápido às áreas de emprego, documentação, saúde e perfil.

O sistema possui duas variações dessa tela: uma para o usuário imigrante e outra para empresas. Cada versão apresenta funcionalidades específicas de acordo com o tipo de usuário, garantindo uma navegação mais adequada às suas necessidades.

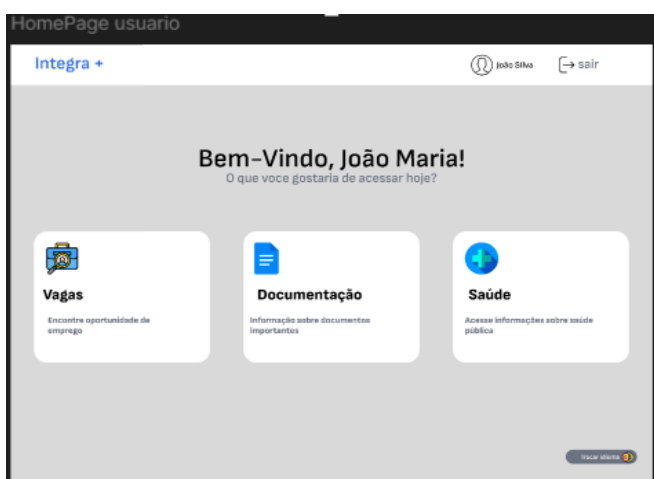


Figura 6 tela principal usuário

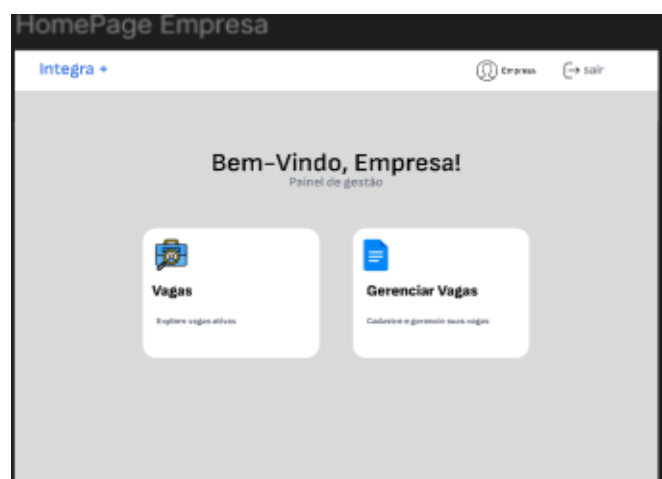


Figura 5 tela principal empresa

Funcionalidade: Busca de Vagas

Permite o usuário visualizar as vagas de emprego disponíveis, e então marcar como favorita ou currículo enviado.

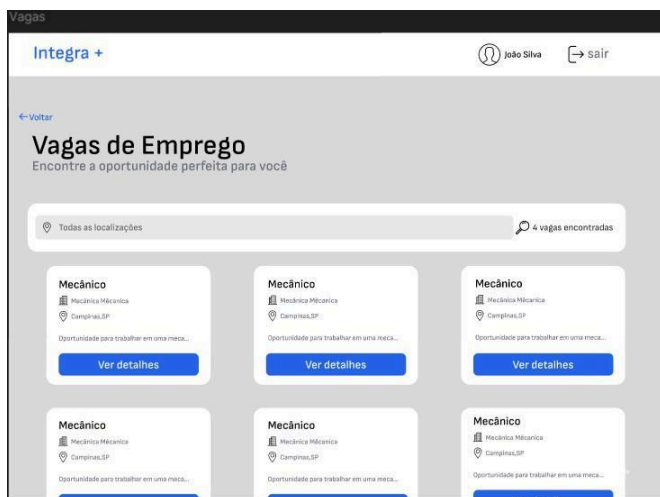


Figura 7 tela de vagas disponíveis

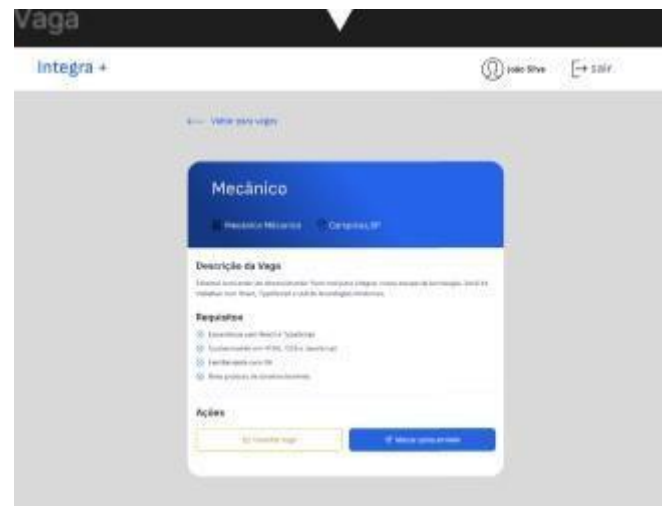


Figura 8 tela da vaga

Funcionalidade: Busca de Vagas

Para empresas, é possível gerenciar as vagas cadastradas, bem como realizar o cadastro de novas oportunidades de emprego.

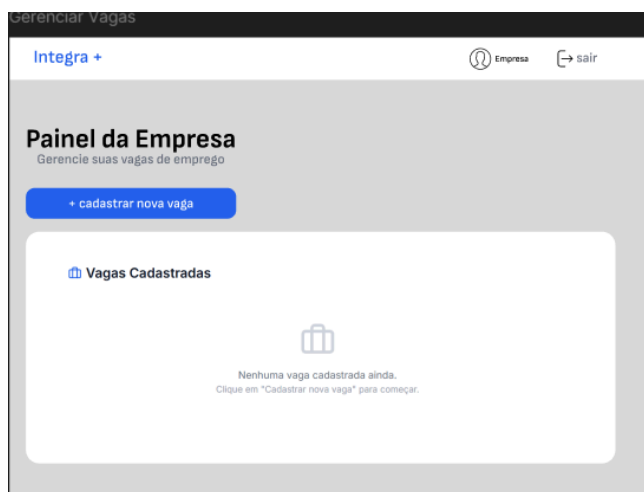


Figura 9 painel vagas empresa

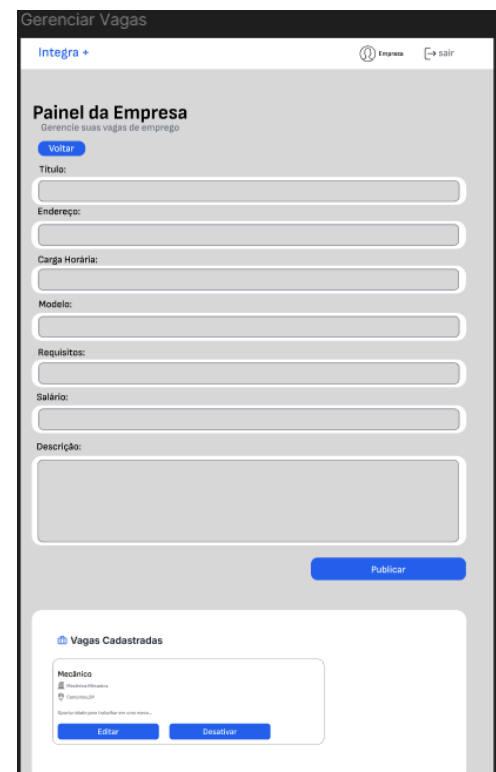


Figura 10 publicar vaga

Funcionalidade: Documentação

Nessa seção, o usuário pode acessar informações relacionadas à regularização documental.

As informações detalhadas ainda não estão dispostas no protótipo, sendo representadas de forma estrutural para demonstrar a organização da funcionalidade.

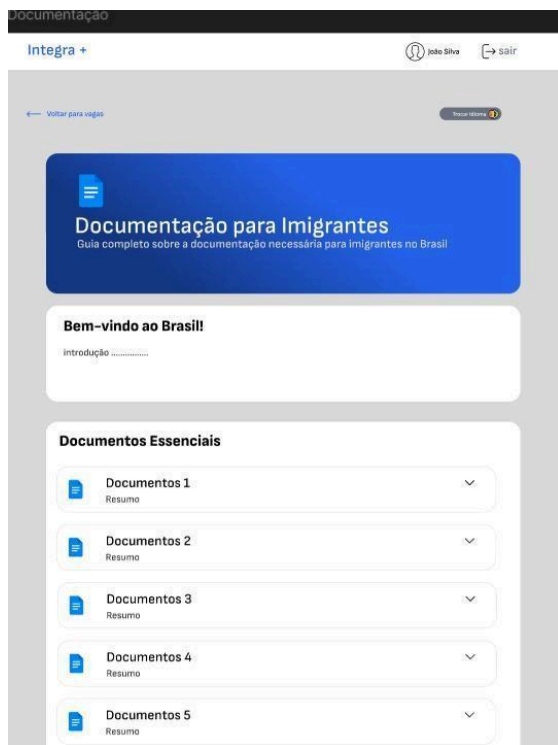


Figura 11 Página de documentação

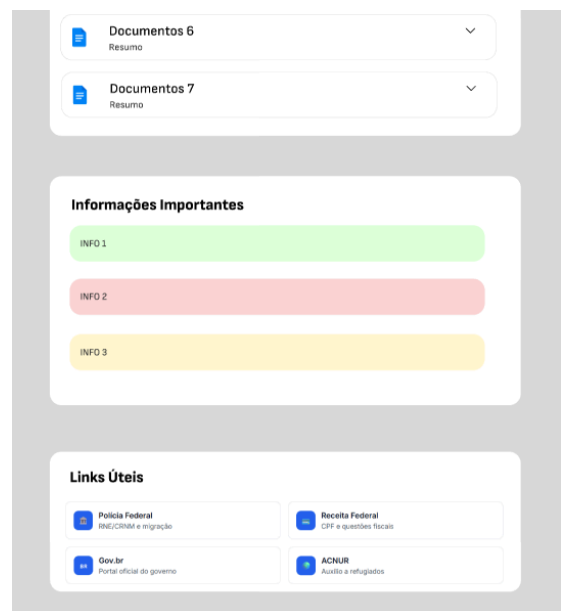


Figura 12 continuação da página de documentação

Funcionalidade: Saúde

Permite ao usuário acessar informações sobre seus direitos à saúde no Brasil, além de possibilitar a busca por unidades de atendimento específicas ou próximas à sua localização.

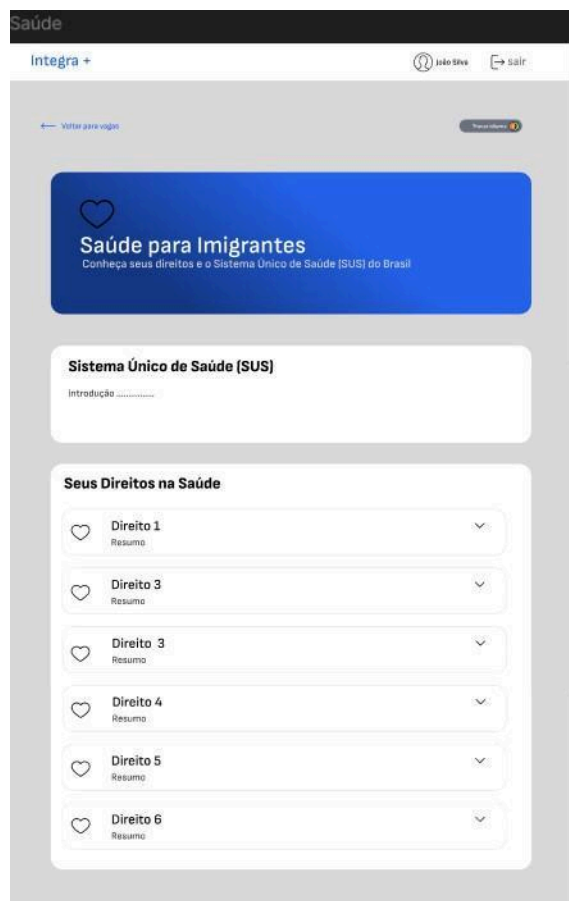


Figura 13 Página saúde

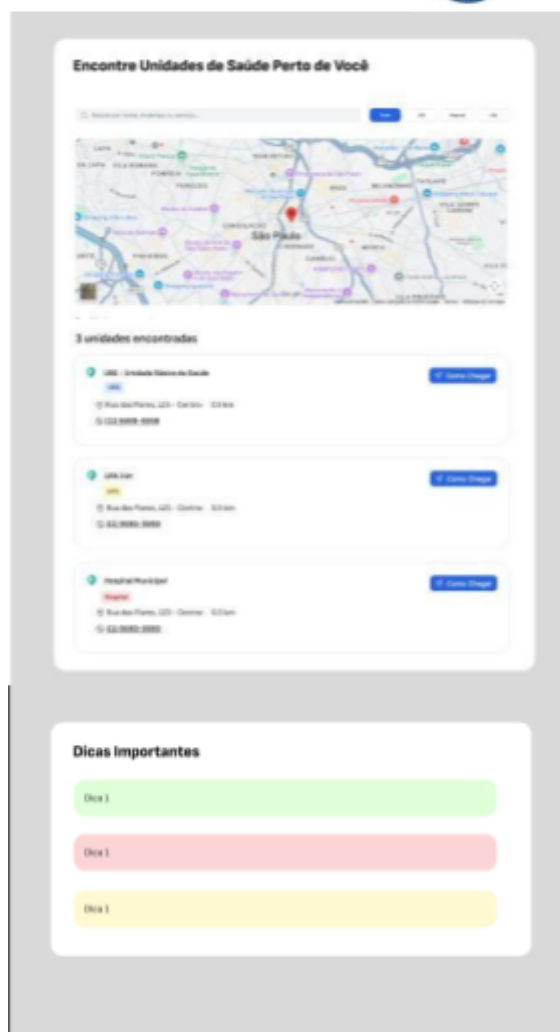


Figura 14 continuação da página saúde

Telas de perfil:

Nessa seção, o usuário ou a empresa pode acessar seu perfil e editar suas informações cadastrais.

No protótipo desenvolvido no Figma, também é possível visualizar as telas de edição de perfil, que permitem a atualização dos dados conforme necessário.

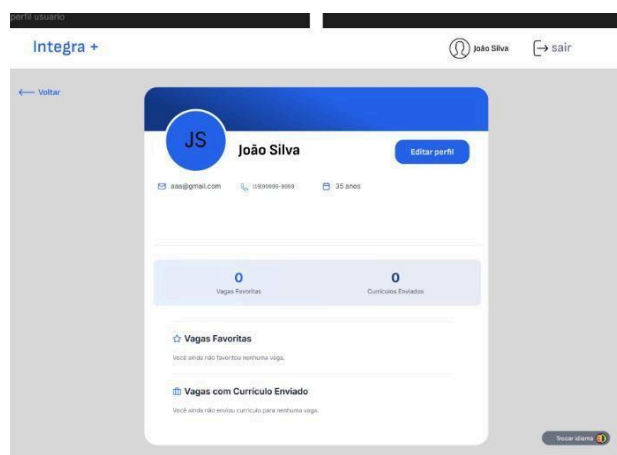


Figura 15 Perfil usuário

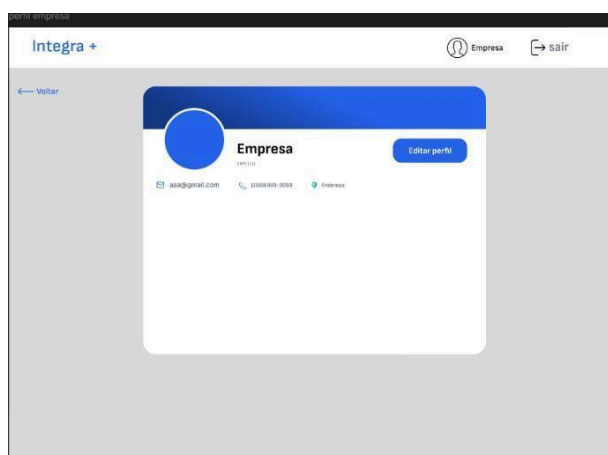


Figura 16 Perfil empresa

2.3 Fluxo básico de navegação

O fluxo básico de navegação do sistema foi projetado de forma simples e intuitiva, permitindo que diferentes tipos de usuários utilizem a plataforma com facilidade.

Inicialmente, o usuário acessa a aplicação e realiza seu cadastro ou login. Após a autenticação, é direcionado à tela principal, onde pode acessar as funcionalidades disponíveis de acordo com seu perfil.

Para usuários imigrantes, o fluxo segue com o acesso às áreas de busca de vagas de trabalho, guia de documentação e serviços de saúde, podendo navegar entre essas seções conforme sua necessidade.

Já para empresas, após o login, o usuário é direcionado a uma versão específica da tela principal, onde pode gerenciar vagas já cadastradas ou publicar novas oportunidades.

Durante a navegação, o usuário também pode acessar a área de perfil para visualizar e editar suas informações cadastrais.

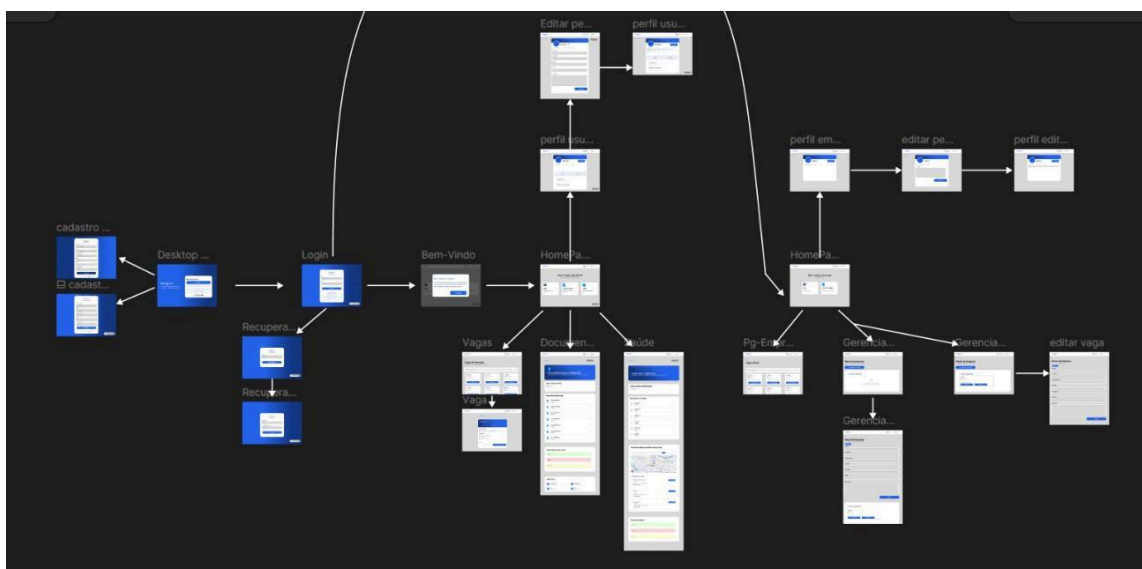


Figura 17 Fluxo de navegação

Identidade visual:

A identidade visual do protótipo foi desenvolvida utilizando dois tons de azul (#153885 e #2563EB), combinados com a cor cinza, com o objetivo de transmitir confiança, clareza e organização.

Além disso, foram utilizados ícones ao longo da interface para tornar a navegação mais intuitiva e facilitar a compreensão das funcionalidades, especialmente para usuários com dificuldade no idioma.

3. Planejamento do Projeto

Para a organização do projeto, está sendo utilizada a plataforma Jira, que permite a aplicação das metodologias Scrum e Kanban.

No Kanban, as atividades foram organizadas em três colunas principais: “Itens Pendentes”, “Em Andamento” e “Concluído”, possibilitando uma visualização clara do progresso das tarefas ao longo do desenvolvimento.

Já no Scrum, o projeto está sendo dividido em sprints, permitindo a entrega incremental das funcionalidades. Atualmente, o grupo encontra-se no fim da Sprint 2, na qual foram desenvolvidas atividades relacionadas à modelagem do sistema e criação do protótipo, incluindo a elaboração do modelo entidade-relacionamento, desenvolvimento das primeiras telas e organização da estrutura do projeto.

As tarefas dessa sprint encontram-se, em sua maioria, concluídas, evidenciando o progresso inicial do desenvolvimento.

Além disso, já foi planejada a Sprint 3, que contempla a continuidade do desenvolvimento da aplicação, com foco na implementação das telas restantes, navegação entre páginas, estrutura do back-end e integração das funcionalidades.

Dentro de cada sprint, as atividades são detalhadas em subtarefas e atribuídas a responsáveis, garantindo uma melhor divisão do trabalho entre os integrantes do grupo.

Dessa forma, a utilização conjunta de Scrum e Kanban contribui para a organização, acompanhamento e viabilidade do projeto, possibilitando a entrega do MVP dentro do prazo estabelecido.

Link de acesso ao Jira: <https://iintegramais.atlassian.net/jira/>

Quadro Kanban

O quadro apresenta a organização das tarefas em diferentes estágios do desenvolvimento, permitindo o acompanhamento do progresso do projeto.

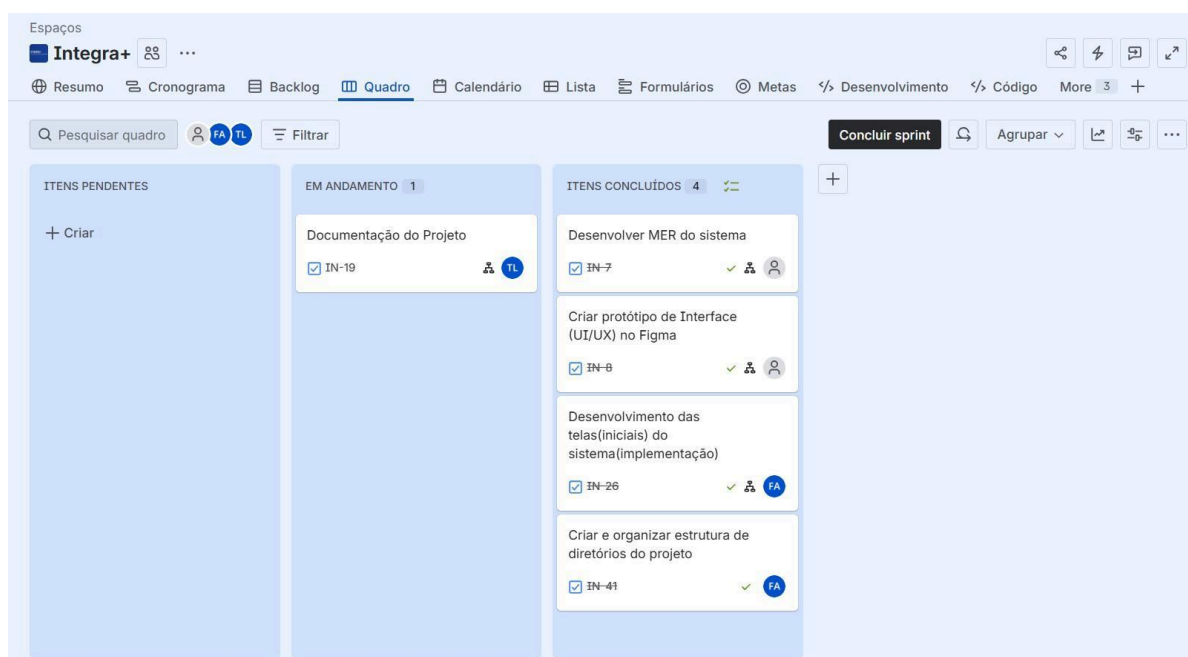


Figura 18 Quadro Kanban

BACKLOG (SPRINTS)

O backlog do projeto evidencia a divisão das atividades em sprints, com tarefas detalhadas e atribuídas aos integrantes do grupo.

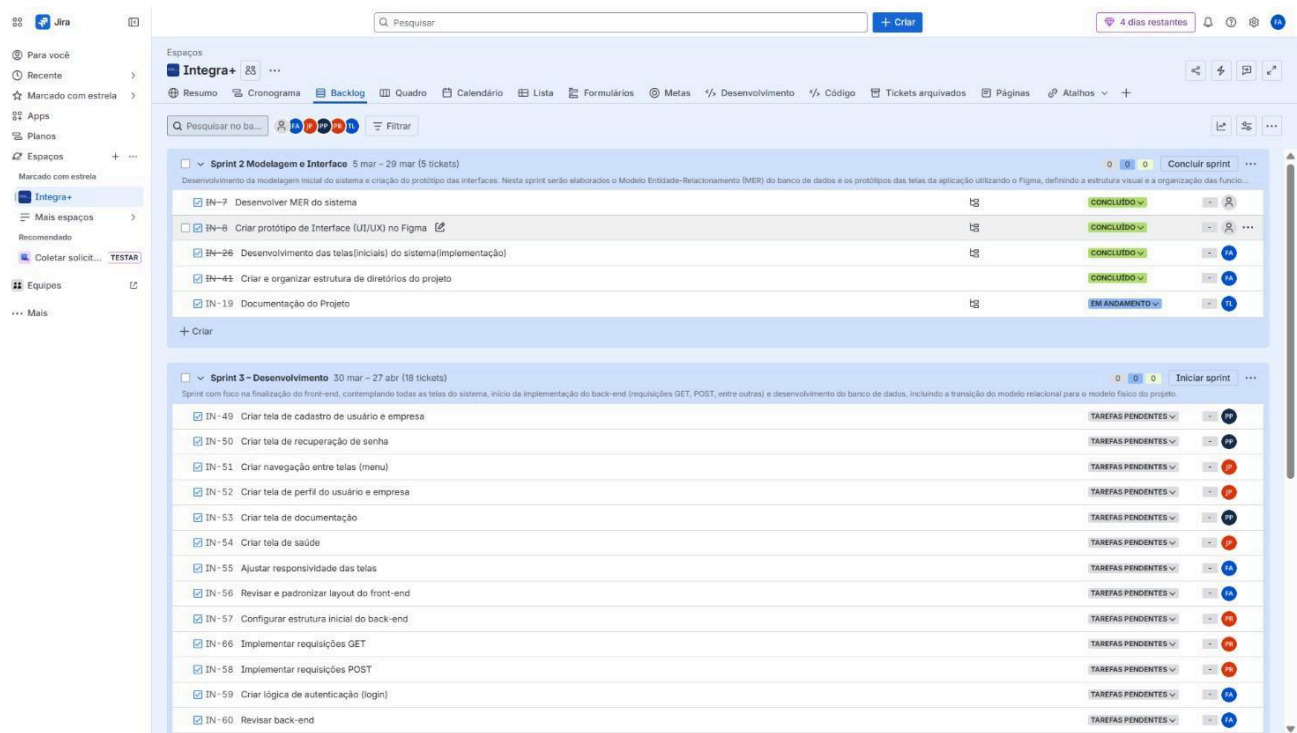


Figura 19 Backlog

CRONOGRAMA

O cronograma apresenta a distribuição das sprints ao longo do tempo, contribuindo para o planejamento das entregas do projeto.

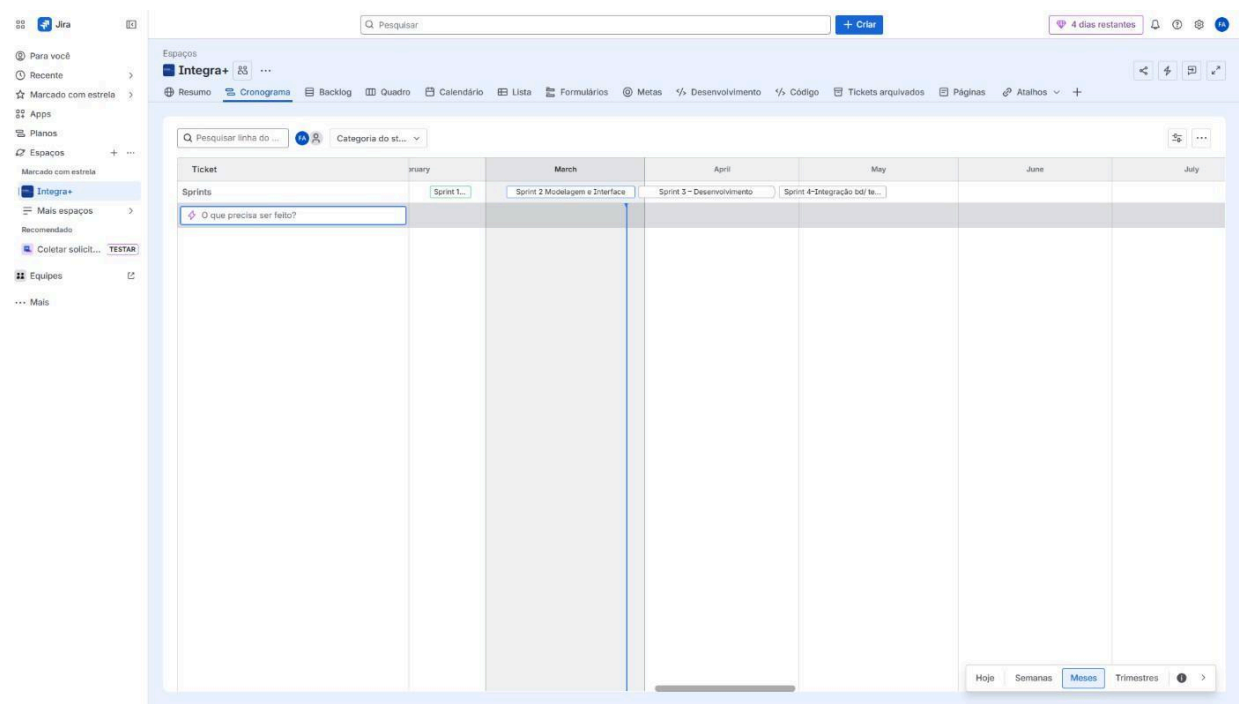


Figura 20 Cronograma

Backlog Inicial do Projeto

O backlog do projeto reúne as principais atividades e funcionalidades previstas para o desenvolvimento do sistema, organizadas de acordo com as prioridades do MVP. Entre os itens definidos, destacam-se:

- Desenvolvimento da interface do usuário (front-end)
- Implementação das funcionalidades de cadastro e login
- Desenvolvimento da busca de vagas de emprego
- Implementação do gerenciamento de vagas por empresas
- Implementação da consulta de unidades de saúde
- Criação e estruturação do banco de dados
- Desenvolvimento do back-end (requisições, APIs e regras de negócio)
- Integração entre front-end, back-end e banco de dados

3.1 Divisão de responsabilidades:

A divisão das tarefas entre os integrantes foi realizada com o objetivo de garantir o cumprimento dos prazos estabelecidos para o desenvolvimento do MVP, bem como a organização e qualidade das entregas.

As atividades foram distribuídas por áreas do projeto: front-end, back-end, banco de dados e documentação. Para o front-end, foram designados dois integrantes responsáveis pela criação das interfaces, e experiência do usuário, sendo estes Pedro Peruchi e José Ricardo. No back-end, dois integrantes atuam no desenvolvimento da lógica da aplicação e das requisições, sendo estes Felipe Amaral e Pedro Pimentel. Já na parte de banco de dados, dois integrantes são responsáveis pela modelagem e estruturação das informações, sendo estes Felipe Amaral e Tiago Aureliano. Por fim, um integrante ficou responsável pela organização e elaboração da documentação do projeto, sendo este Tiago Aureliano.

Além disso, um dos integrantes atua de forma transversal em todas as áreas do projeto, auxiliando na integração entre front-end, back-end e banco de dados, garantindo maior consistência no desenvolvimento e melhor comunicação entre as partes, sendo este Felipe Amaral.

4. LEVANTAMENTO DE REQUISITOS

4.1 Requisitos Funcionais

Os requisitos funcionais representam as principais funcionalidades que o sistema deverá executar para atender às necessidades dos usuários e empresas cadastradas na plataforma.

- **RF01 - Cadastro de usuários**

O sistema deve permitir que imigrantes realizem cadastro na plataforma informando dados básicos, como nome, e-mail e senha.

- **RF02 - Autenticação de usuários**

O sistema deve permitir login seguro para usuários e empresas cadastradas.

- **RF03 - Recuperação de senha**

O sistema deve disponibilizar funcionalidade de recuperação e alteração de senha.

- **RF04 - Busca de vagas de trabalho**

O usuário deverá conseguir pesquisar vagas por cidade, profissão ou região.

- **RF05 - Visualização de detalhes das vagas**

O sistema deverá permitir que o usuário visualize informações detalhadas sobre as oportunidades disponíveis.

- **RF06 - Favoritar vagas**

O usuário poderá marcar vagas como favoritas para acesso posterior.

- **RF07 - Cadastro de empresas**

Empresas deverão conseguir criar contas específicas para publicação de vagas.

- **RF08 - Publicação de vagas**

Empresas cadastradas deverão conseguir publicar novas oportunidades de emprego.

- **RF09 - Gerenciamento de vagas**

Empresas deverão poder visualizar, editar e remover vagas cadastradas.

- **RF10 - Consulta de unidades de saúde**

O sistema deverá permitir a busca de unidades de saúde próximas à localização do usuário.

- **RF11 - Guia de documentação**

O sistema deverá disponibilizar informações organizadas sobre regularização documental e permanência legal no Brasil.

- **RF12 - Edição de perfil**

Usuários e empresas deverão conseguir atualizar suas informações cadastrais.

4.2 Requisitos Não Funcionais

Os requisitos não funcionais representam características relacionadas à qualidade, desempenho, segurança e usabilidade da aplicação.

- **RNF01 - Responsividade**

O sistema deverá funcionar adequadamente em computadores, tablets e dispositivos móveis.

- **RNF02 - Usabilidade**

A interface deverá possuir navegação simples e intuitiva, facilitando o uso por pessoas com pouca familiaridade tecnológica.

- **RNF03 - Segurança**

As senhas dos usuários deverão ser armazenadas de forma criptografada no banco de dados.

- **RNF04 - Desempenho**

O sistema deverá responder às requisições de forma rápida e estável durante a navegação.

- **RNF05 - Compatibilidade**

A aplicação deverá funcionar nos principais navegadores web atuais.

- **RNF06 - Disponibilidade**

O sistema deverá permanecer disponível enquanto o servidor estiver em funcionamento.

- **RNF07 - Organização do código**

O projeto deverá possuir separação entre front-end, back-end e banco de dados, facilitando a manutenção e escalabilidade.

4.3 Regras de Negócio

As regras de negócio definem restrições e comportamentos necessários para o correto funcionamento do sistema.

- **RN01**

O e-mail cadastrado no sistema deverá ser único para cada usuário ou empresa.

- **RN02**

Somente empresas autenticadas poderão publicar vagas de emprego.

- **RN03**

Usuários deverão estar autenticados para favoritar vagas ou acessar funcionalidades personalizadas.

- **RN04**

Cada empresa poderá editar apenas as vagas publicadas por ela mesma.

- **RN05**

O sistema deverá validar campos obrigatórios durante cadastros e autenticação.

- **RN06**

As informações cadastradas deverão ser armazenadas no banco de dados de forma estruturada e consistente.

4.4 Restrições do Sistema

O MVP do projeto apresenta algumas limitações devido ao escopo e prazo da disciplina.

- **RS01**

O sistema depende de conexão com a internet para funcionamento.

- **RS02**

A aplicação será disponibilizada inicialmente apenas em ambiente web.

- **RS03**

O sistema não possui tradução automática em tempo real nesta versão do MVP.

- **RS04**

O envio direto de currículo para vagas ainda não foi implementado.

- **RS05**

O sistema utiliza banco de dados local durante a fase acadêmica de desenvolvimento.

5. Desenvolvimento do Projeto

5.1 Início da Implementação

O projeto iniciou sua construção com a definição da estrutura inicial do sistema e organização do ambiente de desenvolvimento e desenvolvimento da tela inicial e de login.

A aplicação foi desenvolvida utilizando tecnologias web, com separação entre front-end, back-end e banco de dados.

No front-end, foram utilizadas as tecnologias HTML, CSS e JavaScript, responsáveis pela construção da interface do usuário e interação com o sistema.

No back-end, foi utilizado Node.js em conjunto com o framework Express.js, responsável pelo gerenciamento das rotas, processamento das requisições HTTP e implementação das regras de negócio da aplicação.

O banco de dados utilizado no projeto é o MySQL, responsável pelo armazenamento e organização das informações do sistema, como dados de usuários, empresas e vagas de emprego e unidades de saúde.

A estrutura do projeto foi organizada em diretórios específicos para cada camada da aplicação, conforme apresentado a seguir.

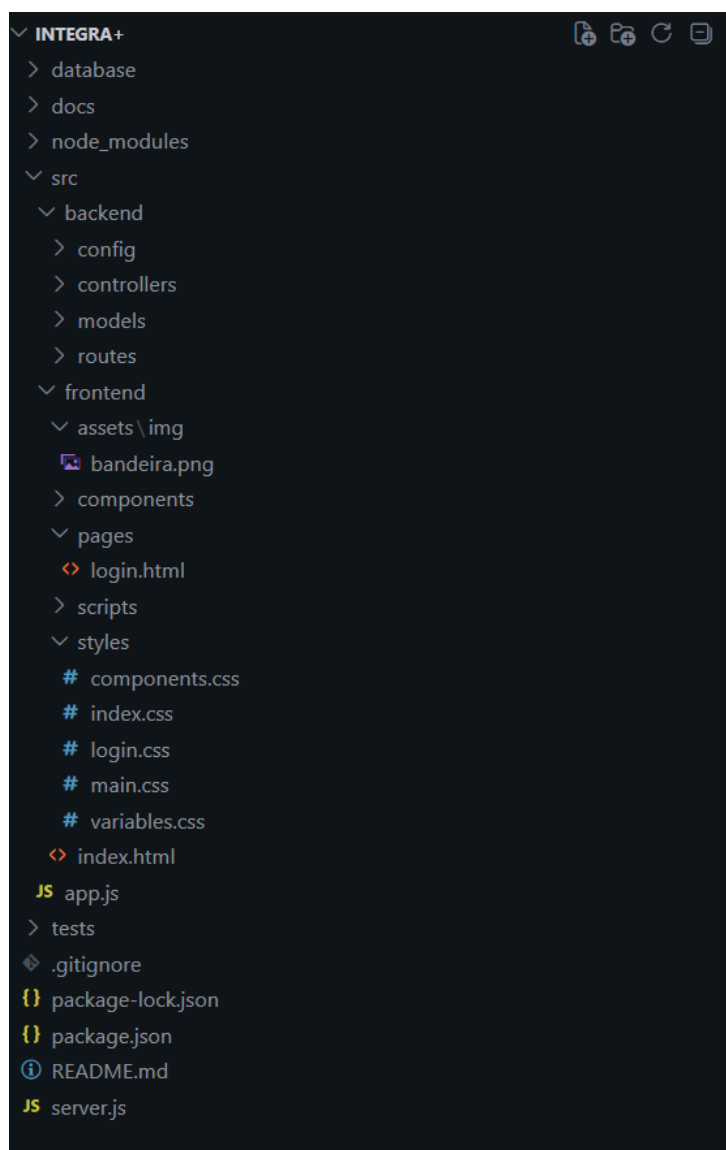


Figura 21 Estrutura inicial do projeto

A estrutura do projeto demonstra a separação entre as camadas do sistema. O diretório “backend” contém a lógica da aplicação, sendo dividido em configurações, controladores, modelos e rotas. Já o diretório “frontend” é responsável pela interface do usuário, contendo páginas, componentes, estilos e scripts.

Além disso, o projeto está sendo versionado utilizando a plataforma GitHub, permitindo o controle de versões, organização do código e colaboração entre os integrantes do grupo por meio de commits e merges.



5.2 Implementação do Front-End

O front-end do sistema foi desenvolvido com base no protótipo elaborado na segunda entrega. Foram utilizadas as tecnologias HTML, CSS e JavaScript para a construção das interfaces.

A navegação entre as páginas segue o modelo definido na etapa de prototipação, garantindo coerência com a proposta inicial do projeto.

O repositório do front-end foi organizado em pastas separadas, contendo *pages*, *scripts* e *styles*, de forma a facilitar a manutenção e a escalabilidade do sistema. Além disso, foram definidas variáveis de estilo para padronização visual da aplicação.

Atualmente, o front-end encontra-se funcional e integrado ao back-end, realizando consumo de dados provenientes da API e do banco de dados.

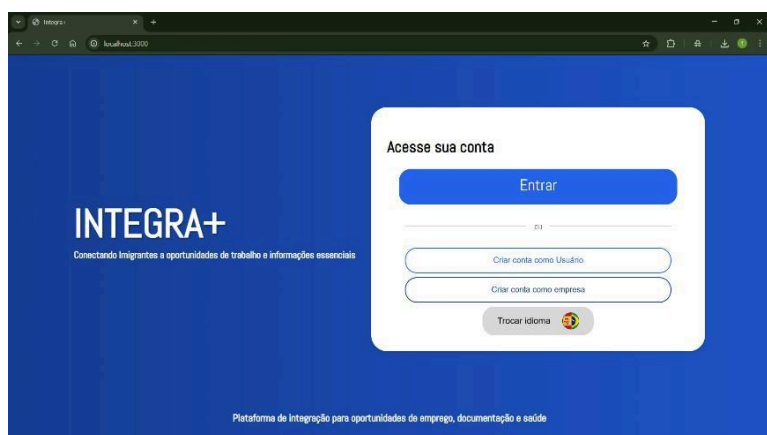


Figura 22:Primeira tela(index)

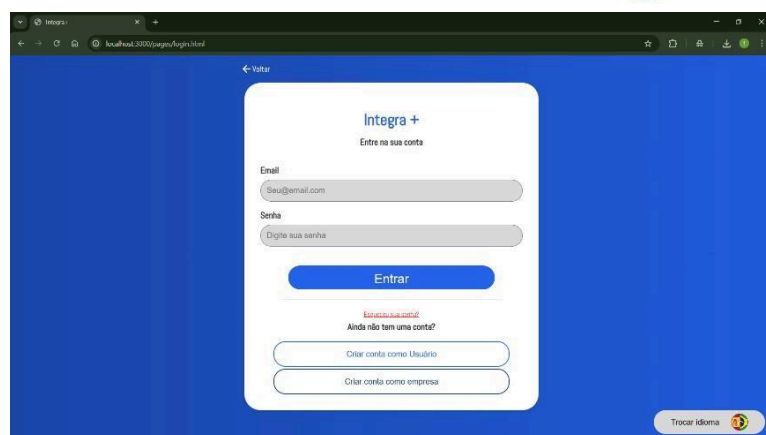


Figura 23:Tela de Login

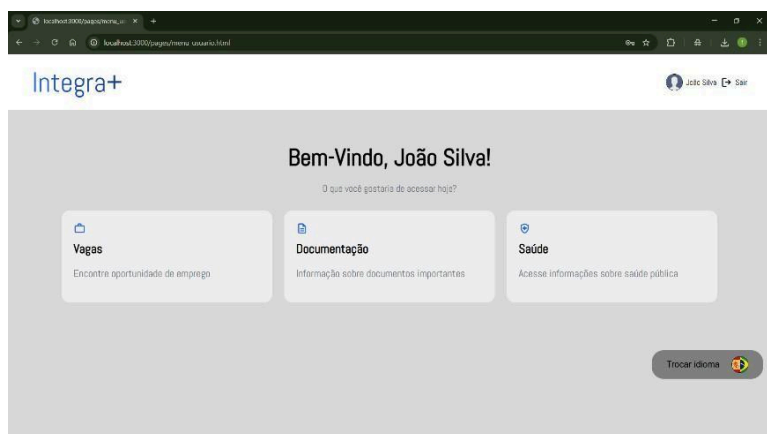


Figura 24:Menu Principal

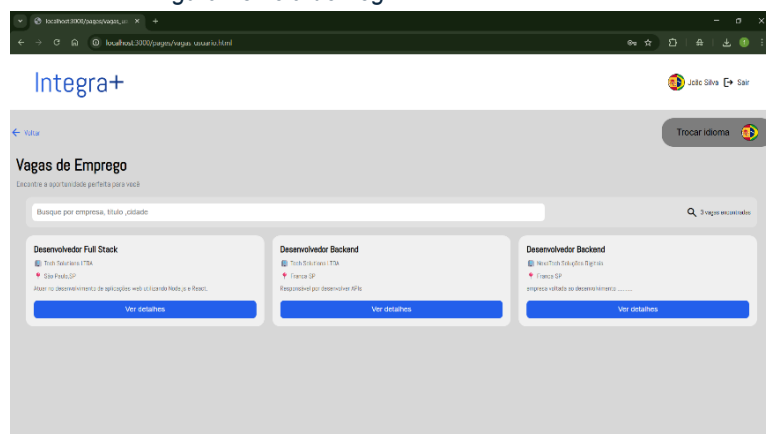


Figura 25:Tela de vagas

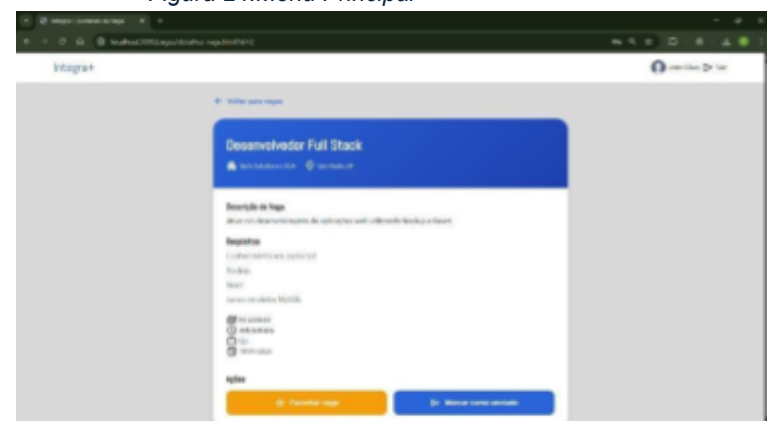


Figura 4: tela de vaga(já consumindo dados do banco

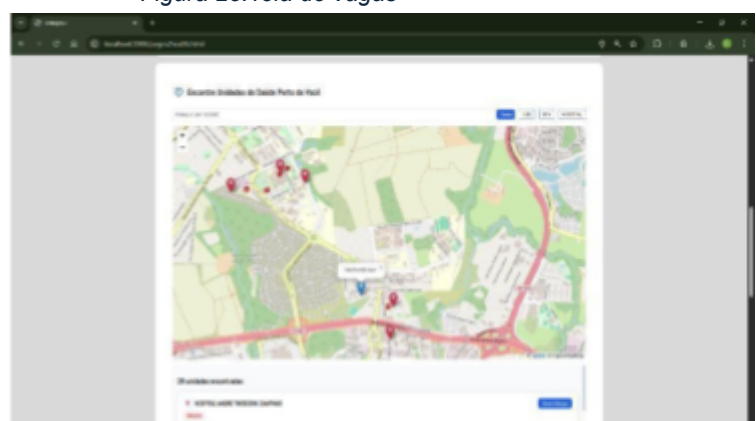


Figura 5:Tela de saúde consumindo dados do

O sistema segue o mesmo fluxo definido na etapa de prototipação, sendo separado em blocos. O primeiro corresponde à área de login, que contempla cadastro de usuário e empresa, além da recuperação de senha.

Após o login, o fluxo se divide entre os perfis de usuário e empresa.

No menu do usuário, é possível visualizar as vagas disponíveis, favoritar oportunidades e acessar informações relacionadas à documentação e saúde. Já no menu da empresa, é possível visualizar todas as vagas cadastradas no sistema, além de cadastrar e editar suas próprias vagas.

5.3 Organização

O front-end do sistema foi estruturado de forma organizada, separando os arquivos em diretórios distintos para facilitar a manutenção e escalabilidade do projeto.

Foram criadas três principais divisões: *HTML*, *scripts* (JavaScript) e *styles* (CSS), permitindo a separação clara entre estrutura, lógica e estilo da aplicação.

A seguir, são apresentados registros visuais da organização dos arquivos no repositório do projeto.

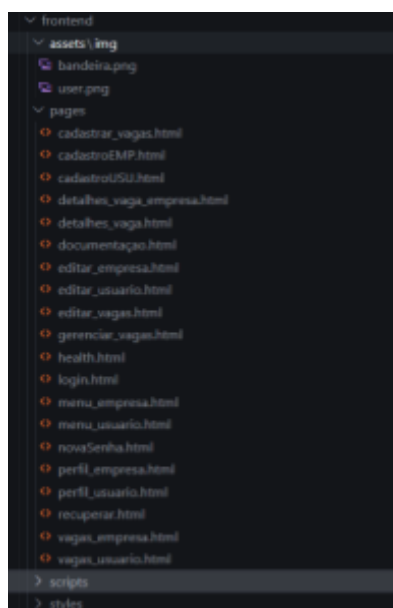


Figura 6: telas do sistema (HTML)

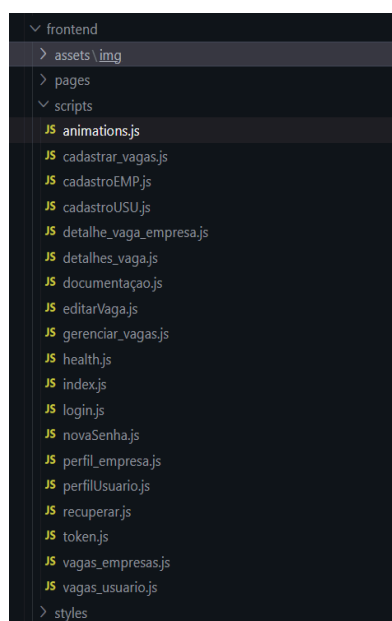


Figura 7: scripts

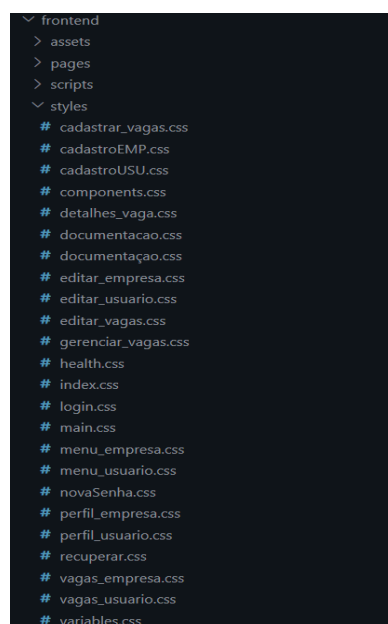


Figura 8: styles

5.4 Back-End em funcionamento

Link repositório GitHub: https://github.com/felipeNamaral/Integra_mais

O back-end foi desenvolvido seguindo uma arquitetura em camadas, composta por *controllers*, *models*, *routes*, *config* e *app/server*. Essa estrutura permite a separação de responsabilidades, facilitando a manutenção e a evolução do sistema.

O sistema conta com mais de 20 rotas implementadas e testadas, utilizando requisições dos tipos GET, POST e DELETE. Essas rotas são responsáveis por operações como consulta, cadastro e remoção de dados, garantindo o funcionamento das principais funcionalidades da aplicação.

A conexão com o banco de dados foi realizada de forma local, com o objetivo de desenvolver e validar o funcionamento do sistema durante o processo de implementação.

Fluxo da API

As requisições no sistema seguem um fluxo estruturado, onde a rota recebe a requisição do front e a direciona para o controller correspondente. O controller processa a lógica da aplicação e interage com o model para comunicação com o banco de dados. Após a execução, a resposta é retornada ao cliente em formato JSON.

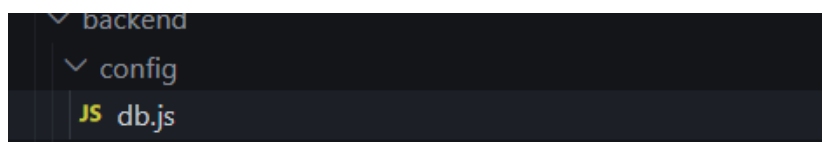


Figura 9:Arquivo de conexão

Conexão

A conexão com o banco de dados foi realizada utilizando o método `createConnection`, de forma local, conectando-se ao banco *integrapius*. Essa conexão permite a comunicação entre o back-end e o banco de dados para execução das operações do sistema.

Exemplo de rotas:

Autenticação:

- /api/login
- /api/cadastro
- /api/usuario

Manipulação das vagas:

- /api/vagas
- /api/vagaById
- /api/vaga(POST)
- /api/vaga (DELETE)

Unidades:

- /api/unidades

5.5 Banco de Dados Finalizado

O modelo entidade-relacionamento (MER/DER) foi atualizado com o objetivo de melhorar a funcionalidade da aplicação e adequar o banco de dados às necessidades do sistema. Como exemplo de alteração, foi adicionada a informação de *cidade* como atributo da entidade vaga, permitindo maior detalhamento das oportunidades cadastradas.

Essas alterações contribuíram para a melhoria da organização dos dados e para uma melhor representação das regras de negócio do sistema.

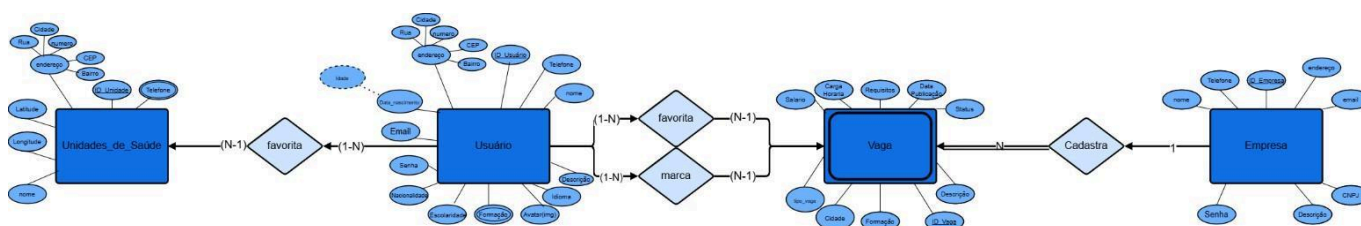


Figura 13: MER atualizado

Link no modelo: <https://drive.google.com/>

Scripts SQL

Para a construção do banco de dados foi utilizado o SGBD MySQL, no qual foram criadas as tabelas necessárias para o funcionamento do sistema. Essas tabelas foram estruturadas com base no modelo entidade-relacionamento (MER), definindo atributos, tipos de dados e restrições como chaves primárias e únicas.

```
CREATE TABLE `usuario` (
  `ID_usuario` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `telefone` varchar(15) DEFAULT NULL,
  `data_nascimento` date DEFAULT NULL,
  `senha` varchar(255) NOT NULL,
  `endereço` varchar(150) DEFAULT NULL,
  `idioma` varchar(50) DEFAULT NULL,
  `nacionalidade` varchar(50) DEFAULT NULL,
  `escolaridade` varchar(50) DEFAULT NULL,
  `formacao` varchar(100) DEFAULT NULL,
  `avatar` varchar(255) DEFAULT NULL,
  `descricao` varchar(500) DEFAULT NULL,
  PRIMARY KEY (`ID_usuario`),
  UNIQUE KEY `email` (`email`)
)
```

Figura 14: Tabela Usuário

```
CREATE TABLE `empresa` (
  `ID_empresa` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `telefone` varchar(15) DEFAULT NULL,
  `endereço` varchar(150) DEFAULT NULL,
  `senha` varchar(255) NOT NULL,
  `cnpj` varchar(20) DEFAULT NULL,
  `descricao` varchar(500) DEFAULT NULL,
  PRIMARY KEY (`ID_empresa`),
  UNIQUE KEY `email` (`email`),
  UNIQUE KEY `cnpj` (`cnpj`)
)
```

Figura 15: Tabela Empresa


```
CREATE TABLE `unidade_de_saude` (
  `ID_Unidade` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(200) DEFAULT NULL,
  `endereço` varchar(150) DEFAULT NULL,
  `telefone` varchar(50) DEFAULT NULL,
  `latitude` decimal(10,7) DEFAULT NULL,
  `longitude` decimal(10,7) DEFAULT NULL,
  `tipo` varchar(100) DEFAULT NULL,
  `nome_municipio` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`ID_Unidade`)
```

Figura 16: tabela Unidades Saúde

```
CREATE
`ID_v
`ID_m
`titu
`de
`sa
`data
`carg
`requ
`tipo
`stat
`cida
PRIMA
KEY `
CONST
CONST
```

Figura 17: Tabela Vaga

Relacionamentos N:N

```
CREATE TABLE `usuario_favorita_vaga` (
  `ID_usuario` int NOT NULL,
  `ID_vaga` int NOT NULL,
  PRIMARY KEY (`ID_usuario`,`ID_vaga`),
  KEY `fk_favorita_vaga` (`ID_vaga`),
  CONSTRAINT `fk_favorita_vaga` FOREIGN KEY (`ID_vaga`) REFERENCES `vaga` (`ID_vaga`) ON DELETE CASCADE,
  CONSTRAINT `usuario_favorita_vaga_ibfk_1` FOREIGN KEY (`ID_usuario`) REFERENCES `usuario` (`ID_usuario`)
```

Figura 18: Relacionamento usuário favorita vaga

```
CREATE TABLE `usuario_favorita_unidade` (
  `ID_usuario` int NOT NULL,
  `ID_Unidade` int NOT NULL,
  PRIMARY KEY (`ID_usuario`,`ID_Unidade`),
  KEY `ID_Unidade` (`ID_Unidade`),
  CONSTRAINT `usuario_favorita_unidade_ibfk_1` FOREIGN KEY (`ID_usuario`) REFERENCES `usuario` (`ID_usuario`),
  CONSTRAINT `usuario_favorita_unidade_ibfk_2` FOREIGN KEY (`ID_Unidade`) REFERENCES `unidade_de_saude` (`ID_Unidade`)
```

Figura 19: usuário favorita unidade de saúde

```
CREATE TABLE `usuario_marca_vaga` (
  `ID_usuario` int NOT NULL,
  `ID_vaga` int NOT NULL,
  PRIMARY KEY (`ID_usuario`,`ID_vaga`),
  KEY `fk_marca_vaga` (`ID_vaga`),
  CONSTRAINT `fk_marca_vaga` FOREIGN KEY (`ID_vaga`) REFERENCES `vaga` (`ID_vaga`) ON DELETE CASCADE,
  CONSTRAINT `usuario_marca_vaga_ibfk_1` FOREIGN KEY (`ID_usuario`) REFERENCES `usuario` (`ID_usuario`)
```

Figura 20: usuário marca vaga

O banco de dados também foi populado com aproximadamente 68 mil registros de unidades de saúde, obtidos por meio de uma base de dados pública. Essa inserção teve como objetivo enriquecer o sistema com dados reais, permitindo testes de busca, filtragem e consulta em maior escala.

5.6 Conexão com o back-end

A integração entre o banco de dados e o back-end é realizada por meio da arquitetura em camadas do sistema. Os controllers recebem as requisições da API e acionam os models, responsáveis por executar as operações de consulta, inserção, atualização e remoção de dados no banco. Dessa forma, o fluxo garante

a comunicação entre a aplicação e o banco de dados de forma organizada e estruturada.

Exemplo de consulta:

```
const getVaga = async (ids) => {
  try {
    const placeholders = ids.map(() => '?').join(',');
    const [rows] = await db.promise().query(
      `SELECT v.*, e.nome AS empresa
      FROM vaga v
      JOIN empresa e ON v.ID_empresa = e.ID_empresa
      WHERE ID_vaga IN (${placeholders})`,
      ids
    );
    return rows;
  } catch (error) {
    console.error("Erro ao obter vaga:", error);
    throw error;
  }
};
```

Figura 21: Exemplo de consulta ao banco

5.7 Integração das Camadas

O sistema já está integrado entre os três principais ambientes: front-end, back-end e banco de dados. Essa integração permite o fluxo completo de dados dentro da aplicação, desde a interação do usuário na interface até o armazenamento e recuperação das informações no banco de dados.

A seguir é apresentado um exemplo de implementação que demonstra essa comunicação entre as camadas do sistema:

```
try {
  const response = await fetch("http://localhost:3000/api/cadastro/usuario", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      nome,
      email,
      dataNascimento,
      senha,
      confirmarSenha
    })
  });
};
```

Figura 22: requisição do front

O front-end realiza a captura dos dados inseridos pelo usuário e os envia para a API do back-end por meio de requisições HTTP. No back-end, o controller recebe essas informações, processa a requisição e aciona o model, que realiza as operações necessárias no banco de dados, como inserção ou consulta de dados.

```
const cadastrarUsuario = async (req, res) => {
  try {
    const {
      nome,
      email,
      dataNascimento,
      senha,
      confirmarSenha
    } = req.body;

    if (!nome || !email || !dataNascimento || !senha || !confirmarSenha) {
      return res.status(400).json({
        sucesso: false,
        mensagem: 'Todos os campos do cadastro de usuário são obrigatórios.'
      });
    }

    if (senha !== confirmarSenha) {
      return res.status(400).json({
        sucesso: false,
        mensagem: 'A confirmação de senha não confere.'
      });
    }
  }
}
```

Figura 23: Controller recebe os dados

```
const existe = await cadastroModel.verificaExistencia(email);

if (existe.length > 0) {
  return res.status(400).json({
    sucesso: false,
    mensagem: 'Email já cadastrado.'
  });
}

const senhaHash = await bcrypt.hash(senha, 10);
await cadastroModel.criarUsuario(nome, email, dataNascimento, senhaHash);

return res.status(201).json({
  sucesso: true,
  mensagem: 'Usuário cadastrado com sucesso.'
});
} catch (error) {
  console.error(error);
  return res.status(500).json({
    sucesso: false,
    mensagem: 'Erro ao cadastrar usuário.',
    erro: error.message
  });
}
```

Figura 24: Criptografia da senha e envio dos dados ao model

```
const criarUsuario = async (nome, email, dataNascimento, senha) => {
  const [result] = await db.promise().query(
    `INSERT INTO usuario
      (nome, email, data_nascimento, senha)
      VALUES (?, ?, ?, ?)`,
    [nome, email, dataNascimento, senha]
  );

  return result;
};
```

Figura 25: Model insere os dados no banco

6. TESTES

6.1 Testes Funcionais

Os testes funcionais foram realizados com o objetivo de verificar se as principais funcionalidades do sistema operam conforme o esperado, garantindo o correto funcionamento dos fluxos definidos no projeto.

Durante os testes, foram validadas funcionalidades como:

- cadastro de usuários;
- autenticação/login;
- cadastro de empresas;
- publicação de vagas;
- busca de vagas;
- consulta de unidades de saúde;
- edição de perfil.

Os testes foram executados tanto pela interface da aplicação quanto através de requisições realizadas no Postman.

Os resultados obtidos demonstraram que as funcionalidades implementadas estavam operando corretamente, permitindo o fluxo esperado entre front-end, back-end e banco de dados.

6.2 Testes de Interface

Os testes de interface tiveram como finalidade analisar o comportamento visual e interativo da aplicação, verificando a navegação entre telas, funcionamento de botões, campos de formulário e organização dos elementos da interface.

Foram validados:

- funcionamento dos botões;
- navegação entre páginas;
- campos obrigatórios;
- menus da aplicação;
- disposição visual das informações;
- adaptação da interface em diferentes tamanhos de tela.

Os testes demonstraram que a aplicação apresentou navegação intuitiva e funcionamento adequado dos componentes visuais implementados.

Como complemento, foram identificadas possibilidades futuras de melhoria relacionadas à acessibilidade e experiência do usuário.

6.3 Testes de Usabilidade

Os testes de usabilidade foram realizados com o objetivo de avaliar a facilidade de utilização da plataforma, considerando principalmente o público-alvo do projeto.

Durante os testes, foram analisados:

- clareza das informações;
- facilidade de navegação;
- organização das funcionalidades;
- experiência geral do usuário;
- compreensão visual das telas.

A interface foi desenvolvida buscando simplificar o acesso às funcionalidades principais, utilizando menus organizados, fluxo intuitivo e elementos visuais de fácil compreensão.

Os resultados demonstraram que os usuários conseguem acessar e utilizar as funcionalidades principais da aplicação de maneira simples e objetiva.

6.4 Testes de Validação de Dados

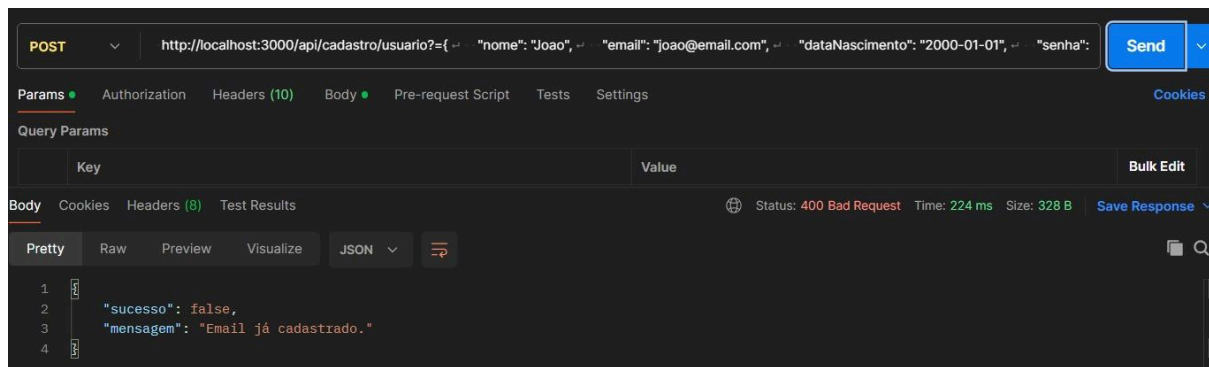
Os testes de validação de dados tiveram como finalidade verificar o tratamento correto das informações enviadas ao sistema, garantindo maior segurança e consistência dos dados armazenados.

Durante os testes, foram verificadas situações como:

- envio de campos vazios;
- tentativa de cadastro com dados inválidos;
- autenticação com senha incorreta;
- validação de campos obrigatórios;
- verificação de dados duplicados.

Os testes foram realizados tanto pela interface do sistema quanto diretamente pelas rotas da API utilizando o Postman.

Os resultados demonstraram que o sistema realiza validações adequadas antes do processamento das informações, reduzindo inconsistências e falhas no cadastro de usuários.



Teste de cadastro de usuário

6.5 Testes de Integração

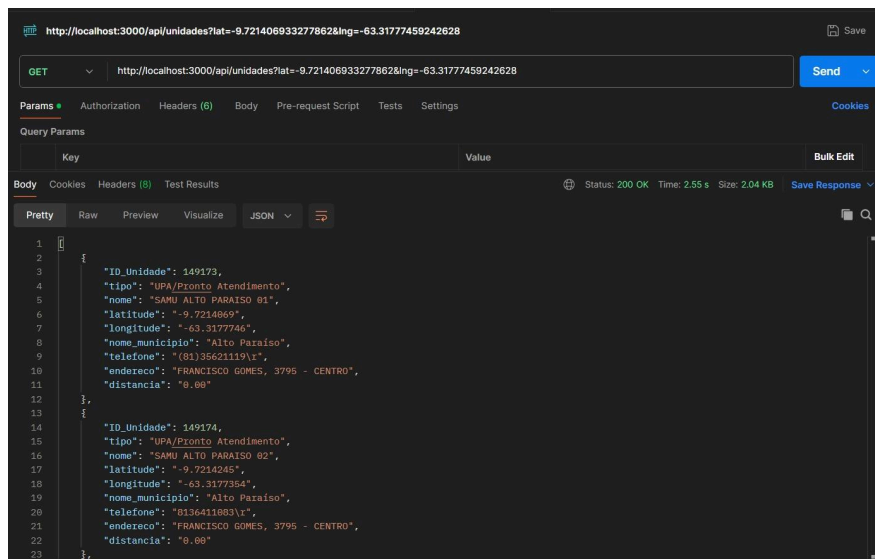
Os testes de integração foram realizados com o objetivo de verificar a comunicação entre as diferentes camadas do sistema, garantindo o funcionamento conjunto entre front-end, back-end e banco de dados.

Durante os testes, foi validado:

- envio de dados do front-end para a API;
- processamento das requisições no back-end;
- comunicação com o banco de dados;
- retorno das informações em formato JSON;
- integração das rotas com controllers e models.

Também foram realizados testes relacionados à busca de unidades de saúde armazenadas no banco de dados hospedado em ambiente cloud.

Os resultados demonstraram que as camadas da aplicação estão integradas corretamente, permitindo o fluxo completo das informações dentro do sistema.



Busca de unidades do banco pelo back-end

6.6 Testes Automatizados

Foram desenvolvidos arquivos específicos para testes automatizados utilizando o Postman, permitindo validar de forma estruturada as principais rotas da aplicação.

Esses testes tiveram como finalidade garantir maior confiabilidade ao sistema e verificar automaticamente o comportamento das requisições HTTP implementadas.

Foram testadas operações como:

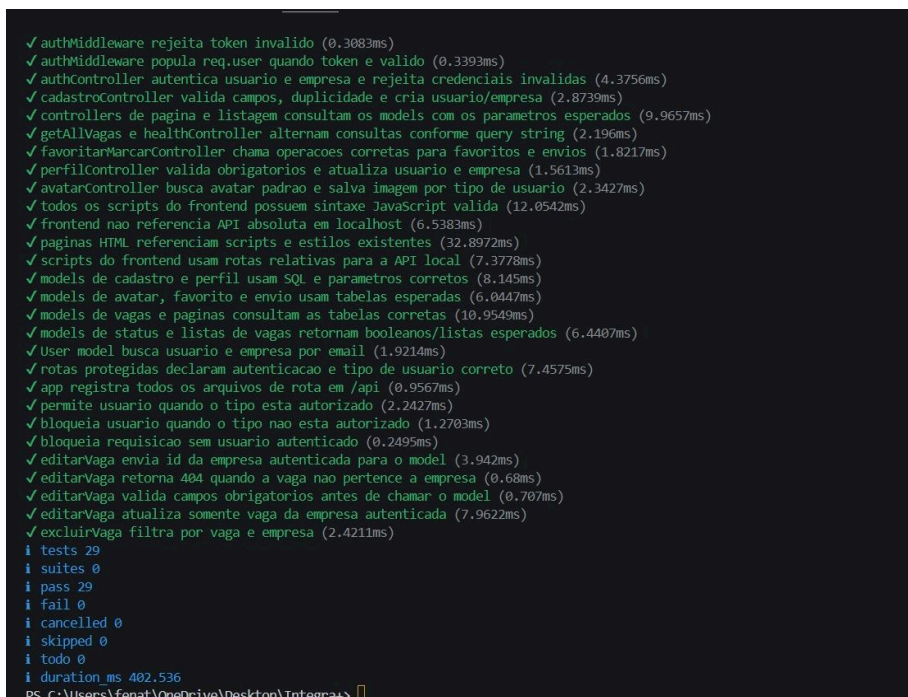
- autenticação;
- cadastro;
- consultas;
- inserção de dados;
- remoção de registros;
- retorno das APIs.

Ao final da execução dos testes automatizados, foram realizados 29 testes no total, obtendo resultado de 0 erros.

Os resultados demonstraram estabilidade e funcionamento adequado das funcionalidades implementadas no sistema.



Testes automatizados do sistema



Resultado dos Testes

6.7 Problemas Identificados e Correções

Durante a execução dos testes, foram identificados alguns problemas relacionados à autenticação, integração entre rotas e validação de informações cadastradas.

Após análise, foram realizadas:

- correções no processamento das requisições;
- ajustes nas rotas da API;
- melhorias nas validações de dados;
- correções na comunicação entre as camadas do sistema.

As correções implementadas contribuíram para maior estabilidade, segurança e confiabilidade da aplicação.

7. IMPLANTAÇÃO

7.1 Execução do Sistema

O sistema INTEGRA+ foi desenvolvido para ambiente web, permitindo acesso por meio de navegadores em computadores e dispositivos móveis. Durante a fase acadêmica do projeto, a aplicação foi executada em ambiente local para fins de desenvolvimento, testes e validação das funcionalidades implementadas.

Para execução do sistema, inicialmente é necessária a instalação das dependências do projeto utilizando o Node.js. Após isso, o servidor back-end pode ser iniciado localmente, permitindo o funcionamento das rotas da API e a comunicação com o banco de dados.

O front-end da aplicação é executado diretamente no navegador, realizando comunicação com o back-end por meio de requisições HTTP.

7.2 Ambiente Necessário

Para funcionamento adequado do sistema, são necessários os seguintes recursos:

- Node.js para execução do back-end;
- MySQL para armazenamento das informações;
- navegador web atualizado;
- ambiente local de execução para a API;
- conexão com a internet para acesso às funcionalidades online.

O sistema foi desenvolvido utilizando Node.js juntamente com o framework Express.js no back-end, responsável pelo gerenciamento das rotas, processamento das requisições e aplicação das regras de negócio.

Já o banco de dados MySQL é responsável pelo armazenamento das informações relacionadas aos usuários, empresas, vagas de emprego e unidades de saúde cadastradas na plataforma.

7.3 Estrutura de Funcionamento

A aplicação foi organizada em três principais camadas: front-end, back-end e banco de dados.

O front-end é responsável pela interface visual e interação com o usuário, permitindo acesso às funcionalidades do sistema por meio das páginas desenvolvidas em HTML, CSS e JavaScript.

O back-end realiza o processamento das requisições recebidas da interface, executando validações, regras de negócio e comunicação com o banco de dados através da arquitetura em camadas composta por routes, controllers e models.

O banco de dados MySQL é responsável pelo armazenamento e recuperação das informações utilizadas pela aplicação.

A comunicação entre as camadas ocorre por meio de APIs HTTP desenvolvidas com Express.js, permitindo o fluxo completo de informações entre interface, servidor e banco de dados.

7.4 Integração e Comunicação da Aplicação

Durante a execução do sistema, o usuário realiza interações diretamente pela interface da aplicação. Essas informações são enviadas ao back-end através de requisições HTTP, onde os controllers processam os dados e acionam os models responsáveis pelas operações no banco de dados.

Após o processamento das informações, o sistema retorna respostas em formato JSON para o front-end, permitindo atualização dinâmica da interface e funcionamento adequado das funcionalidades implementadas.

Essa estrutura contribui para maior organização do código, manutenção da aplicação e escalabilidade do sistema.

7.5 Disponibilização da Aplicação

Nesta etapa acadêmica, o sistema encontra-se executado em ambiente local para fins de desenvolvimento e testes. O código-fonte do projeto está armazenado no GitHub, permitindo versionamento, controle de alterações e colaboração entre os integrantes do grupo.

A utilização do GitHub também contribuiu para maior organização do desenvolvimento, possibilitando acompanhamento das modificações realizadas durante as sprints do projeto.

Em versões futuras, a aplicação poderá ser disponibilizada em ambiente cloud, permitindo acesso público ao sistema, maior disponibilidade da aplicação e possibilidade de expansão para maior quantidade de usuários simultâneos.

Além disso, futuras implementações poderão incluir integração com serviços de hospedagem, bancos de dados remotos e infraestrutura escalável em nuvem.

8. MANUTENÇÃO / EVOLUÇÕES FUTURAS

O projeto INTEGRA+ foi desenvolvido inicialmente como um MVP (Minimum Viable Product), priorizando as funcionalidades essenciais para auxiliar imigrantes no acesso à empregabilidade, documentação e serviços de saúde. Entretanto, a estrutura da aplicação foi planejada de forma organizada e escalável, permitindo futuras expansões e melhorias no sistema.

As evoluções futuras têm como objetivo ampliar a acessibilidade, segurança, desempenho e quantidade de funcionalidades disponíveis na plataforma, proporcionando uma experiência mais completa aos usuários.

Entre as principais melhorias previstas para futuras versões do sistema, destacam-se:

- **Implementação de tradução automática multilíngue**

A plataforma poderá incluir tradução automática em tempo real para diferentes idiomas, especialmente espanhol, facilitando o acesso de usuários que possuem dificuldade com a língua portuguesa e reduzindo barreiras linguísticas durante a navegação.

- **Candidatura direta às vagas pelo sistema**

Em versões futuras, os usuários poderão enviar currículos e realizar candidaturas diretamente pela plataforma, centralizando o processo de busca de emprego em um único ambiente.

- **Sistema de notificações automáticas**

O sistema poderá enviar notificações relacionadas a novas vagas de emprego,

atualizações cadastrais, alterações em processos documentais e informações importantes ao usuário.

- **Sistema de mensagens entre empresas e usuários**

Será possível implementar um canal de comunicação interno entre empresas e candidatos, permitindo maior interação durante processos seletivos.

- **Melhorias de acessibilidade**

Futuras versões poderão incluir recursos voltados à acessibilidade, como aumento de contraste, leitores de tela, ajustes de fonte e melhorias de navegação para usuários com deficiência visual ou dificuldades de utilização.

- **Publicação em ambiente cloud**

O sistema poderá ser hospedado em serviços de computação em nuvem, permitindo maior disponibilidade, desempenho e escalabilidade da aplicação.

- **Desenvolvimento de aplicativo mobile**

Além da versão web, futuras versões poderão incluir um aplicativo mobile para Android e iOS, ampliando o acesso à plataforma em dispositivos móveis.

- **Autenticação com múltiplos fatores**

Para aumentar a segurança da aplicação, poderá ser implementado um sistema de autenticação em duas etapas, reduzindo riscos relacionados ao acesso indevido às contas dos usuários.

- **Ampliação das informações sobre documentação migratória**

O sistema poderá disponibilizar conteúdos mais completos relacionados à regularização migratória, incluindo orientações atualizadas, novos documentos e integração com serviços governamentais.

Além das funcionalidades previstas, futuras versões também poderão incluir integração com APIs externas, sistemas governamentais e serviços públicos, permitindo automatização de consultas e maior quantidade de informações disponíveis aos usuários.

A arquitetura utilizada no desenvolvimento do projeto foi organizada em camadas independentes de front-end, back-end e banco de dados, permitindo maior facilidade para manutenção, atualização e evolução contínua da aplicação.

Dessa forma, o MVP desenvolvido representa uma base sólida para futuras expansões, possibilitando que o sistema evolua gradualmente conforme novas necessidades forem identificadas ao longo de sua utilização.

9. CONSIDERAÇÕES FINAIS

O projeto INTEGRA+ foi desenvolvido com o objetivo de auxiliar imigrantes sul-americanos em situação de vulnerabilidade social no processo de integração à sociedade brasileira, oferecendo uma plataforma voltada ao acesso à empregabilidade, informações documentais e serviços de saúde pública.

Ao longo do desenvolvimento do projeto, foram aplicados conhecimentos relacionados a desenvolvimento front-end, back-end, banco de dados, modelagem de sistemas, APIs, integração entre camadas e metodologias ágeis, permitindo a construção de um MVP funcional e estruturado.

A aplicação foi desenvolvida utilizando arquitetura organizada em camadas, separando front-end, back-end e banco de dados, o que contribuiu para maior organização, manutenção e escalabilidade do sistema. Além disso, foram implementadas funcionalidades essenciais como autenticação de usuários, cadastro de empresas, gerenciamento de vagas, consulta de unidades de saúde e integração completa entre interface, API e banco de dados.

Durante o processo de desenvolvimento, o grupo enfrentou desafios relacionados principalmente à integração entre as diferentes partes do sistema, validação de dados, organização das rotas da API e estruturação do banco de dados. Entretanto, essas dificuldades contribuíram significativamente para o aprendizado técnico dos integrantes e para o amadurecimento do projeto ao longo das sprints realizadas.

Os testes executados durante o desenvolvimento permitiram validar o funcionamento das funcionalidades implementadas, garantindo maior estabilidade, segurança e confiabilidade à aplicação. Foram realizados testes funcionais, testes de integração, validação de dados e testes automatizados utilizando o Postman, possibilitando identificar falhas e aplicar correções importantes durante a construção do sistema.

Além da evolução técnica, o projeto também proporcionou importante crescimento acadêmico e profissional aos integrantes do grupo, contribuindo para o desenvolvimento de habilidades relacionadas à programação, organização de projetos, trabalho em equipe, versionamento de código, metodologias ágeis e resolução de problemas.

O MVP desenvolvido demonstrou a viabilidade técnica da solução proposta, permitindo a construção de uma plataforma funcional, organizada e preparada para futuras expansões. A estrutura implementada possibilita a continuidade do desenvolvimento do sistema, incluindo novas funcionalidades, melhorias de acessibilidade, integração com serviços externos e futura disponibilização em ambiente cloud.

Por fim, o INTEGRA+ busca gerar impacto social positivo ao promover maior inclusão, autonomia e acesso à informação para imigrantes que enfrentam dificuldades de adaptação no Brasil, utilizando a tecnologia como ferramenta de apoio social e integração.